

Here is the revised prompt for a **shared global reference database** that can power time-zone tools, currency converters, vacation planners, meeting planners, country tools, and future location-based utilities.

 Edit



Build a production-ready global reference database and API platform for a date, time, location, currency, travel, converter, and planner application.

The database must act as a reusable global knowledge layer rather than a database built for only one tool. It should support:

- World clocks
- Time-zone converters
- Meeting planners
- Vacation planners
- Travel planning tools
- Currency converters
- Cost-of-travel estimators
- Country and city comparison tools
- Holiday calendars
- Business-day calculators
- Date and time converters
- Sunrise and sunset tools
- Weather-aware planners
- Distance and travel-time tools
- International calling-code tools
- Unit and measurement converters
- Country-specific formatting tools
- Future city-, country-, and time-zone-based utilities

Use PostgreSQL with PostGIS and pg_trgm unless there is a strong technical reason to use another database.

Use reliable, versioned data sources such as:

- GeoNames for countries, administrative divisions, cities, alternate names, coordinates, and populations
- IANA Time Zone Database for time-zone identifiers, aliases, daylight-saving rules, and historical transitions

- Unicode CLDR for localized country, region, currency, language, date, number, and unit names
- ISO 3166 for countries and subdivisions
- ISO 4217 for currencies
- ISO 639 for languages
- Natural Earth for geographic boundaries and map data
- Public holiday providers or official government sources
- Exchange-rate providers for live and historical currency rates
- Weather and sunrise/sunset providers only as external enrichment services

Do not store volatile information such as current weather or live exchange rates directly inside static geographic records. Keep stable reference data separate from frequently changing operational data.

Architectural objective

Design the database as a shared global context platform with these layers:

1. Geographic reference data
2. Administrative hierarchy
3. Place names and localization
4. Time-zone reference and rules
5. Currency reference and exchange rates
6. Language and locale metadata
7. Holiday and business-calendar data
8. Travel and vacation-planning metadata
9. Unit and formatting metadata
10. Source lineage, versioning, and data-quality management

All tools must reference the same canonical country, region, city, currency, language, and time-zone identifiers.

Core database entities

Create normalized tables for:

1. countries
2. administrative_regions
3. cities
4. place_names

5. place_relationships
6. place_redirects
7. time_zones
8. time_zone_aliases
9. city_time_zones
10. country_time_zones
11. currencies
12. country_currencies
13. exchange_rate_providers
14. exchange_rates
15. languages
16. country_languages
17. locales
18. holidays
19. holiday_rules
20. business_calendars
21. weekends
22. seasons
23. climate_summaries
24. airports
25. country_calling_codes
26. measurement_systems
27. units
28. unit_conversions
29. date_time_formats
30. country_travel_profiles
31. travel_advisory_references
32. data_sources
33. import_history
34. source_record_mappings
35. data_quality_issues

Countries

Each country must include:

- Internal canonical country ID
- ISO 3166 alpha-2 code
- ISO 3166 alpha-3 code
- Numeric ISO code
- Official name

- Common name
- Localized names
- Native names
- Country type
- Sovereignty or parent territory where applicable
- Capital city ID
- Continent
- Subregion
- Latitude and longitude
- Geographic boundary reference
- Default currency
- Supported currencies
- Default language
- Official languages
- Common languages
- Calling code
- Country-code top-level domain
- Measurement system
- Driving side
- Weekend definition
- Default date format
- Default time format
- First day of week
- Active status
- Source version
- Created and updated timestamps

Support countries, dependent territories, overseas regions, special administrative regions, and disputed or configurable display labels.

Administrative regions

Support multiple levels of administrative hierarchy:

- State
- Province
- Territory
- Region
- County
- District
- Prefecture

- Department
- Governorate
- Municipality
- Borough
- Other country-specific administrative units

Each administrative region must include:

- Canonical region ID
- Country ID
- Parent region ID
- Administrative level
- Region type
- Official name
- Localized names
- ISO subdivision code where available
- Coordinates
- Boundary reference
- Capital city ID
- Active or historical status
- Source information

Do not assume every country uses states or provinces.

Cities and places

Each city must include:

- Internal canonical place ID
- Official name
- ASCII name
- Native name
- Country ID
- Primary administrative region
- Secondary administrative region where available
- District or county where available
- Latitude
- Longitude
- Population
- Population reference year
- Capital classification
- Place type

- GeoNames ID or other external source ID
- Primary IANA time-zone ID
- Elevation where available
- Active, historical, abandoned, merged, or superseded status
- Created and updated timestamps
- Source version

Support these city and location edge cases:

- Same city name in multiple countries
- Same city name in multiple states or provinces
- City and region sharing the same name
- Multiple cities forming one metropolitan area
- Independent cities and surrounding counties
- Cities crossing administrative boundaries
- Cities near country or time-zone boundaries
- Neighborhoods versus cities
- Airports serving multiple cities
- Historical city names
- Renamed cities
- Merged municipalities
- Duplicate provider records
- Missing population or administrative data
- Cities represented by different coordinates across providers

Never deduplicate places only by name.

Use canonical place IDs, external source IDs, coordinates, country, administrative hierarchy, feature type, and source mappings.

Alternate names and localization

Create a `place_names` table that supports:

- Canonical place ID
- Name
- Normalized name
- ASCII name
- Language code
- Script
- Name type
- Official name

- Short name
- Abbreviation
- Colloquial name
- Transliteration
- Historical name
- Search alias
- Is preferred
- Is historical
- Valid-from date
- Valid-to date
- Source

Search normalization must:

- Apply Unicode normalization
- Support lowercase and case-insensitive matching
- Remove unnecessary repeated whitespace
- Support accent-insensitive matching
- Preserve official display names
- Support native-language scripts
- Support transliterations
- Support apostrophe and hyphen variations
- Support common abbreviations
- Support historical aliases
- Support exact, prefix, token, alternate-name, and fuzzy matching

For a search such as “Hyderabad,” return separate canonical results such as:

- Hyderabad, Telangana, India
- Hyderabad, Sindh, Pakistan

Each result must include enough administrative context to distinguish it.

Time-zone model

Store IANA time-zone identifiers rather than fixed UTC offsets.

Support:

- Multiple cities sharing one IANA time zone

- Countries with multiple time zones
- Half-hour offsets
- Quarter-hour offsets
- Daylight-saving transitions
- Historical offset changes
- Deprecated IANA aliases
- Ambiguous local times
- Nonexistent local times
- International Date Line differences
- Time-zone boundary changes
- Countries that change DST policies

Each time-zone record should include:

- IANA time-zone ID
- Canonical IANA ID
- Alias status
- Standard abbreviation where appropriate
- TZDB source version
- Representative coordinates
- Countries using the zone
- Active status

Do not permanently store a city's current UTC offset. Calculate the offset for the requested timestamp using the IANA time-zone rules.

Currency model

Create reusable currency tables based on ISO 4217.

Each currency must include:

- Internal currency ID
- ISO 4217 code
- Numeric code
- Official name
- Localized names
- Currency symbol
- Narrow symbol
- Number of minor units
- Minor-unit name
- Formatting pattern

- Decimal separator
- Grouping separator
- Active status
- Valid-from date
- Valid-to date
- Replacement currency where applicable

Create a country_currencies relationship to support:

- One country using one currency
- One currency used by multiple countries
- Countries accepting multiple currencies
- Official versus commonly accepted currencies
- Historical currencies
- Currency unions
- Territories using another country's currency
- Currency validity periods

Examples:

- EUR used by multiple countries
- USD used officially or unofficially outside the United States
- Countries with historical currency replacements
- Territories using a parent country's currency

Exchange rates

Keep currency metadata separate from exchange-rate data.

Create exchange-rate tables that support:

- Provider
- Base currency
- Quote currency
- Rate
- Effective timestamp
- Published timestamp
- Rate type
- Live rate
- Daily reference rate
- Historical rate
- Buy rate
- Sell rate

- Mid-market rate
- Provider source
- Data quality status

Store normalized exchange rates and support cross-rate calculations.

Do not assume that all providers update at the same time or use the same rate methodology.

Support:

- Live conversion
- Historical conversion
- Daily reference rates
- Missing weekend rates
- Market holidays
- Inverse rates
- Cross rates
- Provider fallback
- Stale-rate detection
- Currency redenomination
- Inactive currencies
- Decimal precision rules

Vacation and travel-planner model

Design reusable travel-planning metadata without turning the geographic database into a travel-booking system.

Create country_travel_profiles and optional city_travel_profiles with:

- Recommended travel months
- General season classification
- Peak season
- Shoulder season
- Low season
- Approximate climate category
- Average temperature references
- Rainy season indicators
- Public-holiday impact
- Typical weekend days
- Common languages

- Primary currency
- Accepted currencies
- Time zone
- Calling code
- Measurement system
- Driving side
- Visa-reference URL or provider key
- Travel advisory provider reference
- Emergency-number references
- Data source and last-reviewed date

Store climate and seasonal summaries as versioned reference data, not as live weather.

Vacation-planner use cases should include:

- Best time to visit a city or country
- Local time at the destination
- Time difference from the traveler's home city
- Currency conversion
- Local currency and payment information
- Public holidays during the trip
- Weekend and business-day information
- Sunrise and sunset lookup
- Expected seasonal conditions
- Calling codes
- Local date, time, and number formatting
- Travel duration and date-range calculation
- Multi-city itinerary time-zone handling
- Jet-lag and arrival-time calculations
- Budget conversion into destination currency

Holidays and business calendars

Create holiday and business-calendar tables that support:

- Country-wide holidays
- State or province holidays
- City-specific holidays where relevant
- Fixed-date holidays
- Rule-based holidays
- Lunar or religious calendars

- One-time government-declared holidays
- Observed-date rules
- Weekend substitution
- Bank holidays
- Public versus optional holidays
- Business-day calculations
- Holiday names in multiple languages

Each holiday occurrence must retain:

- Country
- Region where applicable
- Holiday date
- Observed date
- Holiday type
- Name
- Localized names
- Rule source
- Data source
- Confidence or verification status

Languages and locales

Support:

- ISO 639 language codes
- Native language names
- Writing scripts
- Right-to-left status
- Official languages by country
- Commonly spoken languages
- Regional languages
- Preferred locale
- Date formatting
- Time formatting
- Number formatting
- Currency formatting
- First day of week
- Weekend definition

Do not assume one language or locale per country.

Unit and converter support

Design unit metadata so the same platform can support converters such as:

- Length
- Area
- Volume
- Weight and mass
- Temperature
- Speed
- Pressure
- Energy
- Power
- Fuel economy
- Data size
- Time duration

Create:

- unit_categories
- units
- unit_aliases
- unit_conversion_rules
- country_unit_preferences

Each unit must include:

- Canonical unit ID
- Unit category
- Symbol
- Localized name
- Base unit
- Conversion factor
- Conversion offset where applicable
- Precision rules
- Valid input aliases

Support affine conversions such as Celsius and Fahrenheit, not only multiplication factors.

Do not store live or location-dependent information in generic unit tables.

Date and time formatting

Create reusable locale and country rules for:

- Date order
- Date separators
- Long-date format
- Short-date format
- 12-hour or 24-hour time
- AM and PM labels
- First day of week
- Week numbering
- Calendar system
- Month names
- Day names
- Ordinal rules
- Date-to-words output
- Time-to-words output

This must support date-to-words and time-to-words tools with localized formal, legal, spoken, and invitation styles.

Place relationships

Create a generic place_relationships table to model:

- City belongs to region
- Region belongs to country
- City is part of metropolitan area
- Airport serves city
- City is near another city
- City replaced another city
- Place has historical predecessor
- Territory belongs to sovereign country
- Capital serves country or region

Avoid hard-coding every relationship into the cities table.

Data freshness classification

Classify data by update frequency.

Mostly static

- Country codes
- City identifiers
- Coordinates
- Administrative hierarchy
- Languages
- Unit definitions
- Currency metadata
- IANA time-zone identifiers

Slowly changing

- City population
- Country currencies
- Administrative boundaries
- Historical names
- Holiday rules
- Travel profiles
- Climate summaries

Frequently changing

- Exchange rates
- Weather
- Travel advisories
- Flight prices
- Hotel prices
- Live events

Frequently changing data must be stored separately, cached with expiration rules, or retrieved from external providers.

Recommended API endpoints

Geography

- GET /countries
- GET /countries/{countryCode}
- GET /countries/{countryCode}/regions
- GET /countries/{countryCode}/cities
- GET /regions/{regionId}
- GET /regions/{regionId}/cities
- GET /cities/{cityId}
- GET /cities/search

- GET /locations/reverse
- GET /places/{placeId}/names

Time zones

- GET /timezones
- GET /timezones/{ianaId}
- GET /timezones/{ianaId}/cities
- GET /time/current
- POST /time/convert
- POST /time/compare
- POST /meetings/find-overlap

Currencies

- GET /currencies
- GET /currencies/{currencyCode}
- GET /countries/{countryCode}/currencies
- GET /exchange-rates/latest
- GET /exchange-rates/historical
- POST /currency/convert

Holidays and planners

- GET /holidays
- GET /countries/{countryCode}/holidays
- GET /regions/{regionId}/holidays
- POST /business-days/calculate
- POST /vacations/plan
- POST /itineraries/time-summary
- GET /destinations/{placeId}/travel-profile

Locales and formats

- GET /languages
- GET /locales
- GET /countries/{countryCode}/formats
- POST /date/format
- POST /date/to-words
- POST /time/to-words

Units and general converters

- GET /unit-categories
- GET /units
- POST /units/convert

City-search endpoint

The city-search endpoint must support:

- q
- countryCode
- regionId
- placeType
- language
- locale
- exact
- fuzzy
- limit
- cursor
- includeHistorical
- includeNeighborhoods
- nearLatitude
- nearLongitude
- timeZone
- currencyCode

Return:

- Stable canonical place ID
- City name
- Matched name
- Match type
- State or province
- Country
- Country code
- Coordinates
- Population
- Place type
- IANA time-zone ID
- Current offset for the requested timestamp
- Primary currency
- Primary language
- Ranking score

Ranking should consider:

- Exact-name match
- Preferred name
- Alternate-name match
- Country and region filters
- User location
- Population
- Capital status
- Search language
- Historical-name penalty
- Fuzzy-match penalty

Data separation requirement

Use one database platform, but separate concerns through schemas.

Recommended PostgreSQL schemas:

- geo
- timezone
- currency
- locale
- calendar
- travel
- units
- source
- audit

For example:

- geo.countries
- geo.regions
- geo.cities
- timezone.zones
- currency.currencies
- currency.exchange_rates
- calendar.holidays
- travel.destination_profiles
- units.units
- source.import_runs

This allows the same database to serve many tools without creating one oversized, tightly coupled schema.

Caching recommendation

Use the database for canonical reference data.

Use Redis, Cloudflare KV, or another cache for:

- Popular city searches
- Current city time
- Exchange-rate responses
- Holiday queries
- Vacation-planner summaries
- Frequently requested conversions

Do not use a cache as the source of truth.

Required automated tests

Generate tests for:

Geography

- Duplicate city names across countries
- Duplicate city names within one country
- City versus administrative-region ambiguity
- Native scripts
- Diacritics
- Transliteration
- Historical names
- Misspellings
- Country filtering
- Region filtering
- Duplicate provider records
- Pagination stability

Time zones

- DST transitions
- Nonexistent local times
- Repeated local times
- Half-hour offsets
- Quarter-hour offsets

- International Date Line
- Historical time-zone changes
- Deprecated aliases
- Multiple zones in one country

Currency

- Same currency across multiple countries
- Country with multiple accepted currencies
- Historical currency
- Missing exchange rate
- Stale rate
- Weekend or holiday rate
- Inverse rate
- Cross-rate conversion
- Decimal precision
- Currency with zero minor units
- Currency with three minor units
- Provider outage

Vacation planner

- Multi-city trip across time zones
- Trip crossing the International Date Line
- Public holiday during travel
- Region-specific holiday
- Destination using a different currency
- Destination with multiple official languages
- Weekend differences
- Seasonal-data gaps
- Missing climate summary
- Travel-advisory provider unavailable

Units and formatting

- Multiplicative conversions
- Temperature conversions using offsets
- Locale-specific number formatting
- Date-format ambiguity
- Right-to-left language formatting
- Date-to-words localization
- Invalid and malicious input

Data quality and governance

Create automated reports for:

- Cities without a country
- Cities without a valid time zone
- Regions without a parent hierarchy
- Invalid IANA identifiers
- Missing currency mappings
- Conflicting currency validity dates
- Duplicate canonical records
- Invalid coordinates
- Missing localized names
- Stale exchange rates
- Holidays without source references
- Travel profiles past their review date
- Invalid unit-conversion rules

Track:

- Source provider
- Source record ID
- Data version
- Import timestamp
- Update timestamp
- Transformation rule
- Confidence level
- Verification status

Delivery phases

Phase 1: global geographic foundation

Build:

- Countries
- Administrative regions
- Cities
- Alternate names
- IANA time zones
- Languages
- Currencies
- Exact and prefix search

- Canonical IDs
- Source versioning

Phase 2: converters and localization

Build:

- Currency conversion model
- Exchange-rate ingestion
- Locale formatting
- Date-to-words
- Time-to-words
- Units and conversion rules
- Fuzzy city search
- Transliteration
- Historical aliases

Phase 3: planner capabilities

Build:

- Holidays
- Business calendars
- Meeting planner
- Vacation-planner reference data
- Seasonal profiles
- Multi-city itinerary time calculations
- Currency-aware trip budgets
- Sunrise and sunset integration

Phase 4: advanced platform capabilities

Build:

- Reverse geocoding
- Geographic boundary queries
- Personalized search ranking
- Advanced travel profiles
- Multiple exchange-rate providers
- Provider reconciliation
- Data-quality dashboards
- Automated source refreshes
- Monitoring and alerting

Final deliverables

Generate:

- Architecture proposal
- PostgreSQL schema
- PostGIS configuration
- SQL migrations
- Entity-relationship diagram
- Index strategy
- Full-text and fuzzy-search strategy
- Import pipelines
- Data normalization rules
- Deduplication process
- Data-source mapping
- REST or GraphQL API specification
- Sample seed data
- Unit tests
- Integration tests
- Data-quality tests
- Cache strategy
- Update and refresh schedules
- Monitoring plan
- Backup and recovery plan
- MVP implementation plan

Keep the first implementation simple and modular.

Prioritize:

1. Stable canonical identifiers
2. Correct administrative hierarchy
3. Correct IANA time-zone mappings
4. Flexible country-to-currency relationships
5. Localized names and formatting
6. Separation of static and volatile data
7. Shared APIs reusable by every converter and planner
8. Source lineage and versioning
9. Search accuracy
10. Future extensibility

Do not build separate copies of country, city, currency, language, or time-zone data for every tool. All tools must use the same canonical global reference database.

A strong implementation structure is **PostgreSQL + PostGIS + pg_trgm**, with separate schemas for geography, time zones, currencies, calendars, travel, locales, and units. This keeps it simple initially while allowing each tool to evolve independently.

Is this conversation helpful so far?

